

Avalia: A Big Data Lens on the Lisbon Real-Estate Market

Daniel Freire Mendes Danish Mujtaba Qureshi Gregor Stefan Sonntag

Big Data Technologies

github.com/danielfmendes/Avalia avalia-frontend.pages.dev

Avalia turns twelve years of Portuguese property listings, plus tourism, search, and macro signals, into parish-level intelligence and twelve-month forecasts for Lisbon.

Contents

1	Introduction	2
2	System architecture and reproducibility	2
3	Data sources	3
4	Forecasting model	3
5	Growth estimate	4
6	The 5 Vs of Big Data: today and at scale	5
7	Discussion and conclusion	5
A	AI prompt log	7

1 Introduction

European housing markets have decoupled: Lisbon, Berlin, and Amsterdam outpace their national averages. Portugal’s INE house-price index is up roughly 50 % since 2019 (Instituto Nacional de Estatística, n.d.), yet no public tool reveals which parish drove it.

Idealista, Imovirtual, and Casa Sapo only show *current* listings. None surfaces parish-level (*freguesia*) price history or the signals that move the market.

Avalia makes these dynamics transparent at the level at which people transact – the parish. Cloudflare D1’s free tier caps writes at **100 000 rows per day** (Cloudflare, 2025), so we scope the prototype to the eighteen Lisbon AML municipalities, where aggregation compresses ~3 M raw rows to ~65 k. The same pipeline extends to other European cities once Lisbon is validated.

Define the project scope through User Stories.

- As a *renter or buyer*, I want parish price history so that I can negotiate from evidence.
- As a *landlord or agent*, I want year-on-year parish benchmarks so that I can price competitively.
- As a *municipal planner*, I want housing-pressure signals so that I can target interventions.

2 System architecture and reproducibility

A renter, an investor, and a municipal planner walk into the same dataset with different questions, so Avalia is split into seven purpose-built pages rather than one catch-all view:

- *Market Overview*: district prices, KPIs, drill-down map.
- *AI Predictions*: 12-month per-region forecast with uncertainty bands.
- *Signals*: price overlaid on Google Trends, dormidas, earnings, construction.
- *Compare*: side-by-side, up to six regions.
- *Rooms*: T0–T4+ trends per typology.
- *Affordability*: budget slider ranking parishes in reach.
- *Time Machine*: monthly scrubber back to 2012.

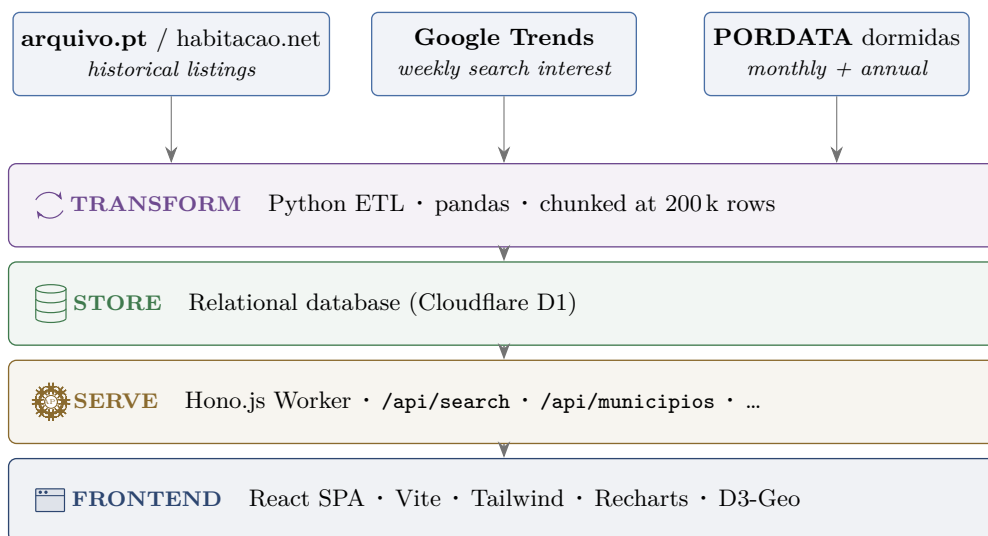


Figure 1: Avalia data pipeline: sources → Python ETL → Cloudflare D1 → Hono.js Worker → React SPA.

The source code, ETL, and schema live in a public GitHub repository at <https://github.com/danielfmendes/Avalia>, and the live demo is publicly accessible at <https://avalia-frontend.pages.dev>.

3 Data sources

The MVP is built on the three Lisbon-scoped feeds listed below. Additional national portals and historical web archives are folded in at Step 2 of the rollout (Section 5) as Avalia extends to other European cities.

- **arquivo.pt + habitacao.net.** *arquivo.pt* (Portugal’s web archive) snapshots Idealista, Imovirtual, and Casa Sapo periodically, and *habitacao.net* mines those snapshots to reconstruct listings history back to 2012 with price, area, rooms, and location per entry.
- **Google Trends.** Weekly Lisbon housing-keyword search interest, a leading indicator: spikes in queries like “comprar casa” precede price moves by one to two months.
- **PORDATA.** Per-municipality series: tourism overnights (dormidas, monthly) as a short-term-rental-pressure proxy, plus four annual macro series (foreign workers, tourist capacity, mean earnings, completed constructions).

Source	Content	Cadence
arquivo.pt + habitacao.net	2.6 GB raw listings, ~3 M transactions	historical
Google Trends	Lisbon search interest	weekly
PORDATA dormidas	tourism overnights per municipality	monthly
PORDATA macro	foreign workers, tourist capacity, mean earnings, completed constructions	annual

Table 1: Data sources and refresh cadences.

4 Forecasting model

Per region, the frontend fits OLS to the trailing 18 monthly €/m² averages and projects 12 months ahead client-side, with the uncertainty band widening 1.5 % per step. The *AI Predictions* page reruns the model per municipality and surfaces a ranked table – sparkline, projected price, expected $\Delta\%$, vs.-district flag, confidence tied to history depth – so a user drills from the district forecast to which municipalities pull it up or down.

Two derived metrics ship alongside. *Rental yield* = (monthly rent $\times$ 12)/sale price on 2023 weighted means, exposed on *Affordability* and *Compare*. *YoY change* is benchmarked only against the calendar months observed in the current period, avoiding spurious dips from incomplete data.

Where exogenous signals fit today. The model extrapolates each region’s own past prices and misses turning points. Google Trends, dormidas, and the PORDATA macro series are rendered on the *Signals* page as overlays for visual correlation against price. Folding them into the forecast itself is left for future work.

Predictor	Key finding	Reference
Airbnb (PT)	1 pp $\uparrow$ Airbnb share $\Rightarrow$ +3.2 % prices in Lisbon/Porto parishes, +32.3 % in touristic parishes (2014–16).	Franco and Santos (2021)
Airbnb (US)	1 % $\uparrow$ listings $\Rightarrow$ +0.026 % prices, with Google search as IV.	Barron et al. (2021)
Google Trends	Local housing search index explains > 40 % of one-month-ahead MSA price variation.	Møller et al. (2024)
News sentiment	Local news sentiment predicts future house prices across US cities.	Soo (2018)
Social sentiment	Twitter activity and public sentiment correlate with housing prices across Manhattan neighbourhoods.	Tan and Guan (2021)
ML method	XGBoost outperforms RF, SVR, MLP, and linear models on house-price regression.	Sharma et al. (2024)
Multi-source	Combining property, amenities, traffic, and emotions beats single-source prediction (28k Beijing properties).	Zhao et al. (2022)

Table 2: Leading-indicator predictors planned for the model.

5 Growth estimate

Two rollout steps stage the geographic expansion. Each step adds a one-shot historical backfill at rollout, then keeps growing from a steady weekly scrape:

- **Step 1** (national rollout: Porto, Algarve, rest of Portugal, 2027): lifts the cumulative footprint from 2.6 to 17 GB stored and from 6 to 31 TB processed.
- **Step 2** (ten major European cities, 2029): lifts it to 58 GB stored and 100 TB processed. Each new country adds its own historical web archive (UK Web Archive, Internet Archive Wayback, BNE), its own forward portal (Rightmove, Immoscout24, SeLogger, Habitaclia, Idealista.it), and an Airbnb scrape per market as a short-term-rental signal complementing the long-term sale and rental listings.

Per-listing footprint (measured on the existing Lisbon dataset):

$$\sigma_{\text{stored}} = \frac{2.6 \text{ GB}}{5.7 \text{ M listings}} \approx 460 \text{ B per listing kept in the aggregated tables},$$

$$\sigma_{\text{processed}} \approx 1.5 \text{ MB per listing fetched (HTML + assets) on each weekly snapshot (HTTP Archive, 2024)}.$$

End-2030 totals. The end-2030 footprint of each region r has two parts: a one-off backfill at rollout, plus a steady monthly increment from the forward scrape. We therefore write

$$S_r = \underbrace{B_r}_{\text{backfill stored (GB)}} + \underbrace{f_r}_{\text{monthly forward rate (GB/month)}} \cdot m_r, \quad P_r = \underbrace{C_r}_{\text{backfill processed (TB)}} + \underbrace{g_r}_{\text{monthly forward rate (TB/month)}} \cdot m_r,$$

where m_r is the number of months the region has been live by end-2030: Lisbon since June 2026 ($m = 55$), the national step since July 2027 ($m = 42$), the EU step since July 2029 ($m = 18$). Summing the three regions:

$$S_{\text{end-2030}} = \underbrace{2.6 + 0.10 \cdot 55}_{\text{Lisbon: 8.1}} + \underbrace{5.6 + 0.22 \cdot 42}_{\text{National: 14.8}} + \underbrace{26.0 + 0.50 \cdot 18}_{\text{EU: 35.0}} \approx 58 \text{ GB},$$

$$P_{\text{end-2030}} = \underbrace{6.0 + 0.18 \cdot 55}_{\text{Lisbon: 15.9}} + \underbrace{9.5 + 0.38 \cdot 42}_{\text{National: 25.5}} + \underbrace{50.0 + 0.50 \cdot 18}_{\text{EU: 59.0}} \approx 100 \text{ TB},$$

giving overall compression $P/S = 100 \text{ TB}/58 \text{ GB} \approx 1730\times$. The Figure 2 gap between line and stack-top from mid-2029 is the European-cities backfill ($1.92\times$ processed/stored, denser than the forward-scrape).

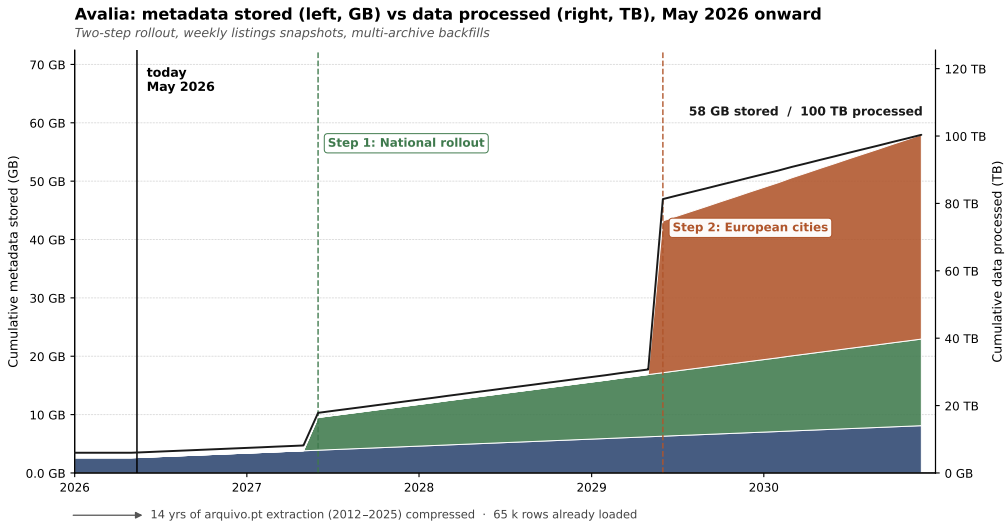


Figure 2: Cumulative metadata stored by region (GB), end-2030.

6 The 5 Vs of Big Data: today and at scale

Why this is a Big Data project. Avalia today extracts 2.6 GB of curated Portuguese listing metadata from roughly 6 TB of *arquivo.pt* page snapshots. Through two rollout steps the production system processes **100 TB of raw archive and weekly portal pages** to extract **58 GB of structured insight** – a $1\,730\times$ compression. *Volume* comes from the touched-data side. *Variety* hardens in Step 2, where each new country brings its own historical web archive and forward portal. *Velocity* is the weekly snapshot cadence, *Veracity* is cross-portal reconciliation, and *Value* is forecast + signals. The five Vs are detailed in the table below.

V	Today	At scale
Volume	2.6 GB stored, 6 TB processed (Lisbon AML, 18 munis)	58 GB stored, 100 TB processed (PT + 10 EU cities)
Velocity	Batch reload only	Weekly <i>Idealista</i> snapshot, same-day re-aggregation
Variety	Four external sources + core listings	+ transit, schools, mortgages, weather, energy certificates
Veracity	Weighted means, chunked dedup, seasonal YoY alignment	Cross-portal reconciliation, anomaly detection
Value	Forecast table, rental-yield, affordability heatmap, signal overlay	Personalised recommendations, probabilistic forecasts, pressure index

Table 3: 5 Vs of Big Data: prototype vs. production scale.

What this needs at scale. The Cloudflare D1 prototype does not survive Step 2. A production build swaps in S3 with Apache Iceberg for the 100 TB archive, Apache Spark or Dask for parallel ETL across snapshots, Kafka for the weekly Idealista feed, and Airflow for orchestration. The serving layer keeps its current shape – only the data plane changes.

7 Discussion and conclusion

Three revenue lines, each backed by a feature already shipped:

- Freemium consumer tier: *Time Machine* and *Affordability* free, with a paid *investor* mode that unlocks the per-municipality forecast table and rental-yield calculator.
- B2B API for banks and real-estate agencies, seeded by the public `/api/search` endpoint (geography $\times$ typology $\times$ area filters, weighted-mean aggregates).
- Anonymised market reports for municipalities, driven by the *Signals* page overlays (tourism, search interest, earnings, construction).

Next business steps: a paid validation pilot with one B2B partner (a small agency or appraisal firm) to anchor API willingness-to-pay, tiered usage-based pricing, and a GDPR / data-licensing review before any municipal contract.

References

- Barron, K., Kung, E., & Proserpio, D. (2021). The effect of home-sharing on house prices and rents: Evidence from Airbnb. *Marketing Science*, 40(1), 23–47. <https://doi.org/10.1287/mksc.2020.1227>
- Cloudflare. (2025). *D1 Platform Limits*. Retrieved May 7, 2026, from <https://developers.cloudflare.com/d1/platform/limits/>
- Franco, S. F., & Santos, C. D. (2021). The impact of Airbnb on residential property values and rents: Evidence from Portugal. *Regional Science and Urban Economics*, 88, 103667. <https://doi.org/10.1016/j.regsciurbeco.2021.103667>
- Fundação Francisco Manuel dos Santos. (n.d.). *PORDATA: Base de Dados Portugal Contemporâneo*. Retrieved May 7, 2026, from <https://www.pordata.pt>
- Fundação para a Ciência e a Tecnologia. (n.d.). *Arquivo.pt: Portuguese Web Archive*. Retrieved May 7, 2026, from <https://arquivo.pt>
- Google. (n.d.). *Google Trends*. Retrieved May 7, 2026, from <https://trends.google.com>
- Habitacao.net. (n.d.). *Dados sobre o mercado imobiliário em portugal*. Retrieved May 7, 2026, from <https://www.habitacao.net/dados>
- HTTP Archive. (2024). *Web Almanac 2024: Page Weight*. Retrieved May 14, 2026, from <https://almanac.httparchive.org/en/2024/page-weight>
- Instituto Nacional de Estatística. (n.d.). *INE: Estatísticas Oficiais de Portugal*. Retrieved May 7, 2026, from <https://www.ine.pt>
- Mendes, D. F., Qureshi, D. M., & Sonntag, G. S. (2026a). *Avalia (source code, public repository)*. <https://github.com/danielfmendes/Avalia>
- Mendes, D. F., Qureshi, D. M., & Sonntag, G. S. (2026b). *Avalia live demo*. <https://avalia-frontend.pages.dev>
- Møller, S. V., Pedersen, T. Q., Schütte, E. C. M., & Timmermann, A. (2024). Search and predictability of prices in the housing market. *Management Science*, 70(1), 415–438. <https://doi.org/10.1287/mnsc.2023.4672>
- Sharma, H., Harsora, H., & Ogunleye, B. (2024). An optimal house price prediction algorithm: XGBoost. *Analytics*, 3(1), 30–45. <https://doi.org/10.3390/analytics3010003>
- Soo, C. K. (2018). Quantifying sentiment with news media across local housing markets. *The Review of Financial Studies*, 31(10), 3689–3719. <https://doi.org/10.1093/rfs/hhy036>
- Tan, M. J., & Guan, C. (2021). Are people happier in locations of high property value? Spatial temporal analytics of activity frequency, public sentiment and housing price using Twitter data. *Applied Geography*, 132, 102474. <https://doi.org/10.1016/j.apgeog.2021.102474>
- Zhao, Y., Ravi, R., Shi, S., Wang, Z., Lam, E. Y., & Zhao, J. (2022). PATE: Property, amenities, traffic and emotions coming together for real estate price prediction. *arXiv preprint arXiv:2209.05471*. <https://doi.org/10.48550/arXiv.2209.05471>

A AI prompt log

A curated, not verbatim, set of prompts that could reconstruct the project's current shape today, grouped by functional area and followed by a data-side problem we hit and the prompt that fixed it.

A.1 Data pipeline (Python ETL)

The pipeline is an offline, run-on-demand step that converts a single curated CSV into the SQL inserts that are then pushed to D1.

```
Write a small Python script that reads csv/summary_lisboa_final.csv and emits batched INSERT INTO habitacao (...) statements into ../backend/sql/Insert_Queries.sql. The CSV columns map 1:1 to the table columns: mes_ano, tipo_venda, tipo_habitacao, quartos, distrito, municipio, freguesia, total_rows, avg_area, avg_preco, avg_m2. Group rows into batches of 100 to avoid SQLITE_TOOBIG when loading into Cloudflare D1. Escape single quotes by doubling them, leave numeric values unquoted, wrap text in single quotes.
```

A.2 Database schema (Cloudflare D1)

A single pre-aggregated table is enough, because aggregation has already been done in the ETL step.

```
Design a Cloudflare D1 schema for aggregated Portuguese housing listings. One table called habitacao, columns: mes_ano (TEXT, "YYYY-MM"), tipo_venda, tipo_habitacao, quartos (TEXT, numeric string like "0.0"), distrito, municipio, freguesia, total_rows (INTEGER, used as a weight when re-aggregating), avg_area, avg_preco, avg_m2 (REAL). Add a composite index on (mes_ano, municipio, freguesia) for the typical filter combination. Drop and recreate the table on each load so the script is idempotent.
```

A.3 Public API (Hono Worker on Cloudflare)

```
Build a Hono Worker for Cloudflare that exposes four GET endpoints backed by a D1 binding called DB: /api/search (main query), /api/municipios (distinct municipalities), /api/freguesias (parishes for a given municipio), /api/health. /api/search accepts level=district|municipality|parish plus municipio, freguesia, tipo_venda, quartos (T0..T4+, where T4+ means quartos >= 4), min_area, max_area. The database stores quartos as a numeric string ("0.0", "1.0", ...), so normalise to T0..T4+ on the way out. Return JSON {success, level, count, data}. CORS allowlist: env.ALLOWED_ORIGIN, http://localhost:5173, http://127.0.0.1:3000.
```

A.4 Frontend dashboard (React + Tailwind + ShadcnUI)

Two prompts: the first lays out the seven-page skeleton, the second produces the Lisbon choropleth that ties the dashboard together.

```
Scaffold a Vite + React 19 + TypeScript + Tailwind v4 + ShadcnUI single-page application with seven lazy-loaded routes: Market Overview, AI Predictions, Signals, Compare, Rooms, Affordability, Time Machine. Add a top navigation bar that switches between them and a dark/light toggle. Share filter state (sale type, room typology, area range, drill-down selection) through a single DashboardContext provider that wraps the routed view. Use Recharts for charts and the project alias @/ for src imports.
```


Write a React component `LisbonMap` that renders the 18 Lisbon Metropolitan Area municipalities as a D3-Geo SVG choropleth. Fit a Mercator projection to the loaded GeoJSON feature collection, fill each region by interpolating a 5-stop green-yellow-red colour ramp over a price metric (EUR/m²), show a hover tooltip with parish name, EUR/m², year-on-year change, and listing count, and on click update the drill-down state so the map re-renders showing only that municipality's parishes. Read the active metric and current selection from the dashboard context.

A.5 Problem encountered: Lisboa-level aggregation

Early in the build the district-level view showed zero rows for Lisbon city. Inspecting the data, Lisboa turned out to be the only municipality whose pre-aggregated “Grouped at Municipio level” marker rows are absent from the source dataset. Every other municipality has them, but for Lisboa only the underlying parish rows exist. A naive `WHERE freguesia = 'Grouped at Municipio level'` therefore filters Lisboa out entirely, leaving a blank tile on the district map. The fix synthesises Lisboa’s municipality row on the fly at query time, weighted by `total_rows`. Cloudflare D1, which is SQLite-based and runs at the edge, does not handle correlated subqueries reliably for this shape, so the implementation uses `UNION ALL`. The resulting branch lives in `api/src/index.ts` under the `level=district` path.

In our D1-backed Hono Worker, `GET /api/search?level=district` returns no rows for Lisbon city. Every other municipality has 'Grouped at Municipio level' rows in the `habitacao` table, but Lisboa has only parish rows. Rewrite the SQL so the district response includes Lisboa, computed on the fly as a weighted aggregate of its parish rows: averages are `SUM(metric * total_rows) / SUM(total_rows)`, and `total_rows` itself is summed. D1's correlated-subquery support is unreliable, so combine the existing district rows with the synthesised Lisboa row using `UNION ALL`. Group the synthesised side by `mes_ano`, `tipo_venda`, `tipo_habitacao`, `quartos`, `distrito`, `municipio`. Order the final result by `mes_ano` ASC.

A.6 Secondary problem: parish-name mismatch on the Lisbon map

A related, smaller area-level issue: the choropleth had blank parishes for many of Lisboa’s *freguesias*. The raw GeoJSON used parish names in slightly different casing, diacritics, and hyphen-versus-space conventions than the `freguesia` strings coming back from `/api/search` (for example *Avenidas Novas* versus *Avenidas-Novas*). The fix was to simplify the GeoJSON and route every name through a single normaliser before comparison.

In `LisbonMap.tsx` the choropleth fill is empty for several parishes because the names in `lisbon-parishes.json` don't exactly match the `freguesia` strings returned by `/api/search`. Both sources use Portuguese with diacritics and inconsistent hyphen-vs-space conventions. Write a `normalizeName` helper that lowercases the input, strips diacritics, and replaces every run of non-alphanumeric characters with a single hyphen, then key the fill lookup by the normalised name on both sides so the map and the API agree.